

Rapport de Projet SAE S31

Lucas Chéron
Bastien Isnard
Noah Dumangin

Difficultés :

La principale difficulté au début du projet a été la répartition des tâches, mais techniquement, le démarrage du projet était plutôt simple. La mise en place des processus, threads et pipes s'est faite sans problèmes.

L'une des premières difficultés a été associée à l'utilisation de signaux entre les threads. En effet un thread ne peut pas recevoir directement un signal, c'est le processus qui le reçoit. Nous avons commencé à utiliser des `pthread_sigmask` pour bloquer les signaux des threads qui n'en avaient pas besoin (ex: `move_and_score` bloque `SIGUSR2`, car c'est `goal` qui est censé les traiter). Finalement, nous utilisons uniquement des `pthread_kill`, qui semblaient faire fonctionner la synchronisation.

A partir de ce moment, les processus étaient correctement synchronisés, une action en menait à une autre, mais il y avait des problèmes majeurs comme des mouvements inattendus de tuiles, un score qui explose ou la fin prématurée du programme. Cela nous a largement bloqués, nous ne trouvions pas l'erreur. C'est le souvenir d'un conseil de M. AYADI qui a débloqué la situation, parlant d'une "gestion de la concurrence d'accès aux ressources". Nous avons ainsi mis en place des mutex pour les ressources partagées entre les threads, c'est-à-dire la variable `game_info` et `last_cmd`. Cela évitait que plusieurs threads accèdent ou modifient au même emplacement mémoire au même moment. Nous avons aussi allégé les handlers de signaux en utilisant les variables volatiles `sig_atomic`.

Mais cela ne fonctionnait toujours pas correctement. Le problème venait seulement d'une incohérence de typage avec `game_logic`, qui utilisait des `int` (32 bits) en mettant à jour la grille, alors que c'était des `uint16_t`. Ce petit problème nous avait donc fait refaire tout notre `game_process` (ajouter les mutex, `sig_atomic`).

Répartition des tâches :

Répartition du travail par fichiers :

Noah :

- **display** : affichage de la grille en fonction des informations reçues.
- **rapport**

Bastien :

- **input_manager** : gère les entrées clavier puis envoie à `game_process`.

- **game_process** : mise en place des threads et du processus display, gestion des pipes et de la synchronisation avec les signaux.
- **main**: met en place input_manager et game process.

Lucas :

- **game_logic**: logique du jeu 2048.
- **game_process** : logique du thread lié à *move_and_score_loop()* (lié à game_logic)

Fonctionnalités:

- Entrée au clavier en utilisant les flèches directionnelles, avec Q ou Ctrl C pour quitter
- Déplacement des tuiles et fusion, augmentation du score
- Apparition d'une nouvelle tuile 2 ou 4 (probabilités : 90% et 10%) à chaque coup valide
- Affichage avec des couleurs de la grille, du score et de l'état de la partie à chaque coup joué
- Gestion de la victoire (une tuile 2048 apparaît)
- Gestion de la défaite (la grille est pleine et aucun coup ne peut être valide)
- Gestion de la sortie de programme (terminaison propre de tous les processus)